Java Interview Questions & Answers

Core Java • Beginner to Interview Level

Topics: Java basics, platform independence, bytecode, JVM/JRE/JDK, data types, type casting, variables, shadowing and access modifiers.

How to use: Read the short answer first, then study the example. In an interview, explain the concept naturally instead of memorizing every sentence.

Memory tip: JDK = develop, JRE = run, JVM = execute bytecode.

1. Java Basics & Platform Independence

What is Java?

Answer: Java is a high-level, object-oriented programming language designed to be portable, secure, and robust. Java source code is compiled into bytecode, which can run on different operating systems using a compatible JVM.

Why is Java platform independent?

Answer: Java is platform independent because Java source code is compiled into platform-neutral bytecode. The bytecode can run on any operating system that has a compatible Java Virtual Machine (JVM).

> **Example / memory tip:** Flow: **.java** → **javac** → **.class bytecode** → **JVM** → **operating system**

What is bytecode?

Answer: Bytecode is the intermediate code generated by the Java compiler from a Java source file. It is stored in a **.class** file and is executed by the JVM.

> **Example / memory tip:** Example: **Hello.java** → compile → **Hello.class**.

What is JVM?

Answer: JVM stands for Java Virtual Machine. It is the runtime engine that loads, verifies, and executes Java bytecode. The JVM itself is platform-specific.

> **Example / memory tip:** Memory trick: **JVM = executes bytecode.**

What is JRE?

Answer: JRE stands for Java Runtime Environment. It provides the components required to run Java applications, including the JVM and runtime libraries.

> **Example / memory tip:** Memory trick: **JRE = run Java.**

What is JDK?

Answer: JDK stands for Java Development Kit. It contains tools required to develop Java programs, including the Java compiler and runtime components.

> **Example / memory tip:** Memory trick: **JDK = develop Java.**

Difference between JDK, JRE and JVM?

Answer: JDK is used to **develop** Java applications. JRE provides what is needed to **run** Java applications. JVM is the component that **executes** Java bytecode.

> **Example / memory tip:** Simple memory: **JDK** → **JRE** → **JVM** conceptually.

What does javac do?

Answer: javac is the Java compiler command. It compiles Java source code in a **.java** file into bytecode stored in a **.class** file.

> **Example / memory tip:** Example: **javac Main.java** → usually creates **Main.class**.

What is a .class file?

Answer: A .class file contains Java bytecode generated by the Java compiler. The JVM loads and executes this bytecode.

> **Example / memory tip:** Example: **Test.java** → compile → **Test.class**.

Why is Java called platform independent?

Answer: Java is called platform independent because compiled bytecode is not tied directly to one operating system. The same bytecode can be executed on Windows, macOS, Linux, and other supported platforms through their JVM implementations.

2. Data Types

How many primitive data types are there?

Answer: Java has **8 primitive data types**.

What are the 8 primitive data types?

Answer: The eight primitive types are: **byte, short, int, long, float, double, char, boolean**.

> **Example / memory tip:** Memory grouping: **byte, short, int, long** = integer types; **float, double** = floating-point; **char** = character; **boolean** = true/false.

What is the difference between primitive and reference types?

Answer: Primitive types represent simple values such as numbers, characters, and boolean values. Reference types refer to objects rather than directly representing the object itself.

> **Example / memory tip:** Example: **int age = 22;** is primitive. **String name = "Pratham";** is a reference type.

Is String primitive?

Answer: No. **String is not primitive.** String is a class in Java, so a String variable is a reference type.

> **Example / memory tip:** The primitive character type is **char**, not String.

What is the default value of an instance int?

Answer: The default value of an instance variable of type **int** is **0**.

What is the default value of a boolean instance variable?

Answer: The default value of an instance variable of type **boolean** is **false**.

Do local variables get default values?

Answer: No. Local variables do not automatically receive default values. A local variable must be assigned a value before it is read.

> **Example / memory tip:** Example: **int x; System.out.println(x);** causes a compile-time error.

3. Type Casting

What is widening?

Answer: Widening conversion means converting a value from a smaller compatible numeric type to a larger numeric type. Java normally performs this automatically.

> **Example / memory tip:** Example: **int x = 10; double y = x;**

What is narrowing?

Answer: Narrowing conversion means converting a larger numeric type to a smaller numeric type. It may lose information and normally requires explicit casting.

> **Example / memory tip:** Example: **double d = 10.5; int x = (int)d;**

Does widening require explicit casting?

Answer: Usually no. Java automatically performs widening conversions for compatible numeric types.

Example / memory tip: Example: **int** → **long** → **double**.

Does narrowing require explicit casting?

Answer: Yes, for normal numeric narrowing conversions, explicit casting is required.

Example / memory tip: Example: **double d = 10.8; int x = (int)d;**

What happens to decimal values during double → int conversion?

Answer: The fractional part is discarded; Java does not round the value during this cast.

Example / memory tip: Example: **(int)10.9 = 10** and **(int)-10.9 = -10**.

What happens when an int value outside byte range is cast to byte?

Answer: The value is narrowed to 8 bits, so the result can wrap around according to Java's two's-complement representation.

Example / memory tip: Example: **int x = 130; byte b = (byte)x;** results in **-126**. Byte range: **-128 to 127**.

4. Variables

What is a local variable?

Answer: A local variable is declared inside a method, constructor, or block and can be used within its scope. It does not receive an automatic default value.

Example / memory tip: Example: **void test() { int x = 10; }**

What is an instance variable?

Answer: An instance variable is a non-static variable declared inside a class but outside methods, constructors, or blocks. Each object gets its own copy.

Example / memory tip: Example: **class Student { String name; }**.

What is a static variable?

Answer: A static variable belongs to the class rather than individual objects. It is normally shared by all objects of that class.

Example / memory tip: Example: **static int count;**.

What is a final variable?

Answer: A final variable can be assigned only once. After assignment, it cannot be reassigned.

Example / memory tip: Example: **final int MAX = 100;**.

What is a static final variable?

Answer: A static final variable is a class-level constant: it is shared by the class and cannot be reassigned after initialization.

Example / memory tip: Example: **static final int MAX_SIZE = 100;**.

Does Java have global variables?

Answer: Java does not have traditional global variables like some languages. Variables are declared within classes, methods, or blocks. A static variable can provide class-level shared state, but it is not a separate global-variable feature.

Difference between static and instance variables?

Answer: An instance variable belongs to each object, so different objects can have different values. A static variable belongs to the class and is shared among instances.

Example / memory tip: Example: **name** can differ for every Student, while **schoolName** could be static.

Can we change a final variable?

Answer: No. Once a final variable has been assigned, it cannot be reassigned. For a final reference, the reference cannot change, although the referenced object may still be mutable.

Can a static variable be changed?

Answer: Yes. A static variable can be changed unless it is also declared final or another restriction prevents reassignment.

Example / memory tip: Example: **static int count = 0;** can be updated with **count++**.

5. Shadowing & this

What is variable shadowing?

Answer: Variable shadowing happens when a variable in a narrower scope has the same name as a variable in an outer or enclosing scope, causing the narrower-scope variable to hide the other variable in that context.

Example / memory tip: Common example: a constructor parameter has the same name as an instance variable.

What is this?

Answer: this is a reference to the current object. It is commonly used to access the current object's instance variables or methods and to distinguish instance variables from parameters with the same name.

Example / memory tip: Example: **this.name = name;** means the instance variable receives the parameter.

How do you access an instance variable when a local variable has the same name?

Answer: Use the **this** keyword before the instance variable name.

Example / memory tip: Example: **Student(String name) { this.name = name; }**.

6. Access Modifiers

What are the four access modifiers?

Answer: The four access levels are **private, default (package-private), protected, and public**.

Difference between private and default?

Answer: private members are accessible only within the same class. **Default/package-private** members are accessible within classes in the same package, but not from unrelated classes in another package.

Difference between protected and public?

Answer: public members have the broadest access when the declaring type is accessible. **protected** provides package access and also permits qualifying subclass access from another package.

Can a private variable be accessed from another class?

Answer: Not directly. A private variable is accessible only inside its declaring class. Another class can access it indirectly through methods such as getters/setters if they are provided.

Example / memory tip: Example: **private int age;** can be exposed through **public int getAge()**.

What happens if no access modifier is specified?

Answer: If no access modifier is specified for a class member, it has **default (package-private)** access. It can be accessed by classes in the same package but not by classes in unrelated packages.

Quick Revision Sheet

Topic	Remember
Java	High-level, object-oriented programming language
Platform independent	Same bytecode can run on a compatible JVM
Bytecode	Intermediate code stored in .class
JVM	Executes bytecode
JRE	Runtime environment
JDK	Development kit; compiler and development tools
javac	.java → .class
Primitive types	8: byte, short, int, long, float, double, char, boolean
String	Reference type, not primitive
Instance int default	0
Instance boolean default	false
Local variable	No automatic default value
Widening	Smaller → larger; usually automatic
Narrowing	Larger → smaller; explicit cast normally required
double → int	Fractional part discarded
final	Cannot be reassigned after assignment
static	Belongs to class / shared
this	Reference to current object
private	Same class only
default	Same package
protected	Same package + qualifying subclass access
public	Broadest access

Interview technique: Give the definition first, then one small example. If the interviewer asks a follow-up, expand your explanation.